Real-Time AI Text Editing via Low-Latency and Memory-Efficient Transformer Inference

Waqas Ahmed
i232540@isb.nu.edu.pk

Huzaifa Khalid
i232508@isb.nu.edu.pk

Abstract

Modern AI-assisted text editors require strict latency guarantees to maintain responsiveness. This work evaluates an inference pipeline designed to minimize time-to-first-token (TTFT) while maintaining memory efficiency on commodity hardware. The proposed system combines editor-aware KV-cache prefetching, speculative decoding, and adaptive KV-cache eviction policies. The evaluation framework measures latency, throughput, and memory statistics under realistic editor workloads derived from Wikipedia revision histories.

1 Introduction

Modern AI-assisted text editors now enable autocomplete, grammar correction, rewriting, and summarization. These features have strict latency constraints. Cursor based autocomplete must start outputting within 200 milliseconds to feel responsive. Existing systems struggle with frequent short requests due to large, dynamic KV-cache and prefill costs.

Time-to-first-token (TTFT) latency is the biggest bottleneck in interactive inference. Current approaches do not adequately address the combination of low latency, memory efficiency, and single-user responsiveness on commodity hardware such as mid-range GPUs (RTX 3060) or consumer CPUs like the (Apple M2).

This project evaluates an inference pipeline that achieves minimal TTFT while remaining memory-efficient and also deployable on standard consumer hardware. The pipeline combines three optimization techniques: editor-aware KV-cache prefetching, speculative decoding with a small draft model, and adaptive KV-cache eviction policies.

Initial experiments conducted in a constrained environment (Google Colab T4) already demonstrate measurable reductions in TTFT using KV-cache prefill, which supports the feasibility of our proposed approach.

2 Literature Review / Related Work

2.1 Speculative Decoding and Prefill Prefetching

The prefill phase dominates TTFT. Speculative decoding introduces a draft-target mechanism where a smaller model proposes tokens and a larger model verifies them in parallel, improving throughput without affecting output quality.

Prompt caching techniques reuse attention states for repeated prefixes. In editor settings, where context changes incrementally, preloading recent tokens into KV-cache during idle time reduces TTFT without modifying decoding.

Draft model selection is very critical, as alignment with the target model's thinking matters more than size. This motivates future exploration of smaller domain-tuned draft models.

2.2 KV-Cache Optimization

KV-cache management is the primary memory bottleneck. Paged Attention reduces fragmentation by treating KV-cache as virtual memory. Quantization (INT8/FP8) reduces memory bandwidth with minimum quality loss for small contexts.

Eviction strategies affect both performance and quality. LRU is simple but ignores token importance. Adaptive eviction retains high-attention tokens but introduces its own overhead.

2.3 Inference Engines

vLLM is designed for high-throughput GPU inference and leverages advanced memory management techniques such as PagedAttention to efficiently handle KV-cache. However, it assumes access to dedicated GPU resources and is primarily optimized for multi-request serving scenarios.

In contrast, llama.cpp supports lightweight deployment with hybrid CPU/GPU execution and aggressive quantization (e.g., INT4/INT8), making it better suited for commodity hardware.

Given our focus on low-latency inference on consumer systems, llama.cpp is selected as the primary inference engine.

2.4 Base Paper Evaluation

Speculative Decoding Techniques. Speculative decoding methods (1; 2; 7) aim to reduce the inefficiency of autoregressive decoding by generating and verifying multiple tokens per step. These approaches increase token-level parallelism and improve throughput compared to single-token decoding. However, their gains are primarily reflected in steady-state throughput rather than Time-to-First-Token (TTFT), which is a key latency metric in interactive systems. They also assume multiple models and GPU parallelism, often in distributed or batched settings, which may introduce overhead in single-user, low-latency environments.

KV-Cache Optimization. KV-cache optimization techniques (8; 3) reduce memory overhead by retaining important tokens while evicting others, enabling stable long-context inference with bounded memory. These methods are mainly designed for streaming and long-sequence scenarios, and do not explicitly target initial latency. In particular, they do not consider editor-aware KV-cache prefetching or cold-start latency, which are important for minimizing TTFT in interactive systems.

Quantization and Memory Efficiency. Quantization-based approaches (4) reduce model size and memory bandwidth requirements, enabling deployment on CPUs and mid-range GPUs. While effective in lowering overall inference cost, they introduce accuracy–efficiency trade-offs and are typically evaluated independently of decoding strategies. They do not directly address the temporal aspects of inference, particularly TTFT.

Attention Optimization. Attention optimization techniques (5) reduce computational complexity compared to dense attention. These methods improve efficiency for long sequences and large batches but do not eliminate the sequential dependency of autoregressive decoding. As a result, their impact on TTFT in single-request settings is limited.

Inference Systems. Modern inference systems (3; 6) improve execution through batching, scheduling, and memory-aware strategies. While effective for multi-user throughput, such designs may introduce coordination overhead that is less suited for applications requiring immediate responsiveness.

From a PDC perspective, existing approaches primarily emphasize throughput-oriented parallelism and distributed execution, while giving less attention to fine-grained latency optimization for single-request workloads.

Overall, prior work improves transformer inference in terms of throughput and scalability but places less emphasis on TTFT, often assumes high-end hardware, and focuses on multi-user scenarios. These observations motivate a unified inference pipeline that targets low-latency, single-user performance through KV-cache prefetching, speculative decoding, and adaptive eviction.

2.5 Benchmarking Methodology

Traditional NLP metrics such as BLEU or ROUGE do not adequately capture user-perceived responsiveness in interactive systems. Instead, this work focuses on system-level latency metrics, particularly Time-to-First-Token (TTFT) and end-to-end latency.

To approximate realistic usage, a workload based on Wikipedia edit sequences will be constructed. These edits simulate common interactive behaviors such as short completions, incremental rewrites, and frequent cursor-triggered requests, enabling evaluation under conditions that reflect real-world editor interactions.

3 Proposed Methodology

3.1 Pipeline Architecture

The proposed system will be organized as a multi-stage inference pipeline designed to minimize TTFT while maintaining memory efficiency.

- **Editor-aware KV-cache prefill:** Relevant context will be prefetched based on cursor position to reduce prefill latency.
- **Speculative decoding:** A lightweight draft model (TinyLlama) will generate candidate tokens, which will be verified by a larger target model (Mistral 7B) to accelerate decoding.
- **Draft-target verification:** Accepted tokens will be streamed immediately, while rejected tokens will trigger fallback to the target model.
- **KV-cache eviction:** Least Recently Used (LRU) and adaptive policies will be applied to manage memory under constrained hardware.

The system will be implemented using FastAPI, with streaming endpoints supporting real-time autocomplete and text rewriting.

3.2 Evaluation Framework

Experiments will be conducted on consumer-grade hardware, specifically targeting CPU-based and low-VRAM environments compatible with llama.cpp (e.g., Apple M2, RTX 3060).

The evaluation will focus on the following metrics:

- **TTFT (P50, P95):** Measures responsiveness, capturing both typical and worst-case latency.
- **Time-per-output-token:** Evaluates steady-state decoding efficiency.
- **Acceptance rate:** Indicates the effectiveness of speculative decoding by measuring the proportion of draft tokens accepted by the target model.
- **Memory usage:** Tracks KV-cache footprint and overall system memory consumption.

Workloads are derived from Wikipedia revision histories and OpenWebText prompts. Baseline comparisons will be performed against standard autoregressive decoding without speculative optimization.

3.3 Preliminary Results

Initial experiments conducted using a quantized Mistral 7B model on a Google Colab T4 GPU environment. We measure Time-to-First-Token (TTFT) under two conditions: cold inference and KV-cache prefill.

Results show that KV-cache prefill reduces TTFT from approximately 4.2 seconds to 2.9 seconds, representing a reduction of nearly 30%. Time-per-output-token (TPOT) remains largely unchanged, confirming that latency improvements primarily arise from reduced prefill computation.

These preliminary findings support the hypothesis that prefill dominates latency in interactive LLM inference and that KV reuse is an effective optimization strategy.

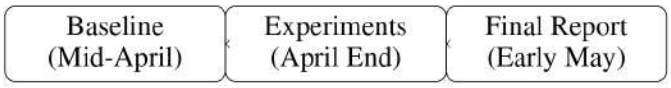
3.4 Experimental Design

Experiment 1: Speculative Decoding Block size $k \in \{1, 4, 8\}$ is evaluated for latency and acceptance trade-offs.

Experiment 2: KV Eviction LRU vs adaptive eviction evaluated under constrained memory.

Experiment 3: Prefill Impact Prefetching is toggled to measure TTFT reduction in simulated typing workloads.

3.5 Implementation Timeline



4 Summary of Design Decisions

- Draft model: TinyLlama 1.1B
- KV eviction: LRU (baseline), Adaptive (experimental)
- Quantization: INT8
- Backend: llama.cpp
- Reference: vLLM
- Prefill: 32 tokens, 150 ms idle timer

5 References

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023.

[2] Z. Chen, Y. May, and A. Rush, "Accelerating Large Language Model Decoding with Verification," arXiv:2305.09781, 2023.

[3] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[4] Y. Sheng *et al.*, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. ICML*, 2023.

[5] A. Shah *et al.*, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low Precision," arXiv:2407.08608, 2024.

[6] G. Gerganov, "llama.cpp: Inference of LLaMA models in pure C/C++," GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[7] S. Kim *et al.*, "Big Little Decoder," in *Proc. NeurIPS*, 2023.

[8] G. Xiao *et al.*, "Efficient Streaming Language Models with Attention Sinks," arXiv:2309.17453, 2023.